Handling Large Blobs

問題在哪裡

學過系統設計的人都知道，影片、圖片這類大型檔案應該放在 S3 這樣的 Blob Storage，而不是資料庫。這個分離讓儲存和運算可以各自獨立擴展，也讓資料庫查詢保持流暢。

為什麼要用 Blob Storage？

資料庫擅長處理有複雜查詢的結構化資料，但非常不擅長大型二進位物件。把一個 100MB 的檔案存成 BLOB，會讓查詢效能、備份時間和 replication 全部崩掉。物件儲存（Object Store）例如 S3 就是為此而生：無限容量、99.999999999%（十一個九）的耐久性、按物件計費。一個粗略的判斷標準是：超過 10MB 且不需要 SQL 查詢的東西，大概就應該放進 Blob Storage。

Blob Storage 解決了儲存問題，但沒解決資料傳輸問題。標準的做法是讓檔案的 bytes 走過你的 application server：客戶端上傳一個 2GB 的影片，API server 接收後再轉送給 Blob Storage；下載也一樣，Blob Storage 傳給 API server，再由 API server 轉給客戶端。這對小檔案沒問題，但檔案越大就越撐不住。

我們強迫 application server 充當一個毫無價值的管道。它對傳輸沒有任何貢獻，只增加延遲和成本。雲端供應商本來就有全球基礎設施、斷點續傳能力和巨大的頻寬，我們卻把自己有限的 application server 插在中間當中間人，憑空製造了一個不必要的瓶頸。

所以解法是：從「透過伺服器上傳」切換到「客戶端直接上傳到 Blob Storage」；下載則從全球 CDN（例如 CloudFront）取得。你的伺服器永遠不碰實際的 bytes。

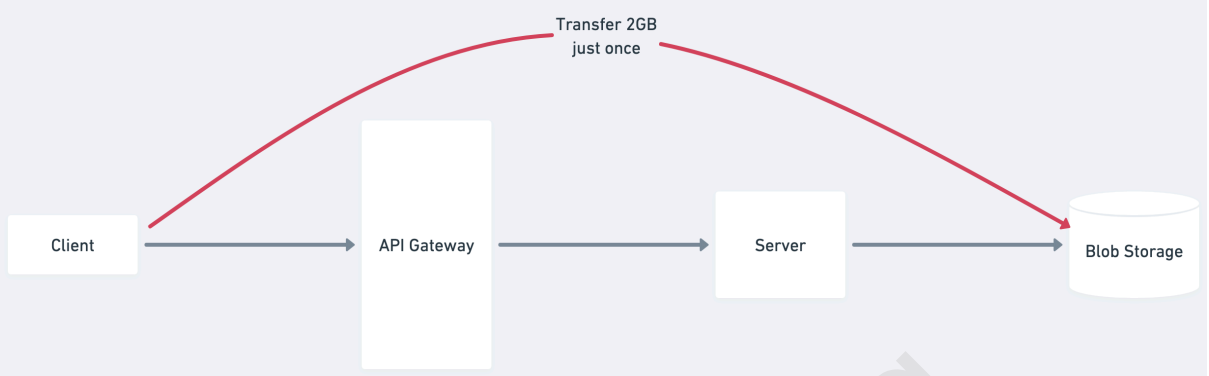
但這樣做本身還不夠，因為新的問題又冒出來了。那個上傳到 99% 才斷線的 25GB 大檔案，意味著得從頭開始。因為上傳完全繞過你的伺服器，你失去了對進度的可視性。資料庫顯示「pending」，S3 裡可能已經有完整的檔案，也可能沒有，你不查根本不知道。

看起來只是搬運 bytes 這件看似簡單的事，在分散式系統裡其實充滿了最終一致性、進度追蹤和狀態同步的挑戰，而這些都是讓一切流過自己伺服器時從沒遇過的。

buildmoat.org

解法

不讓資料流過你的伺服器，而是給客戶端臨時的、有範圍限制的憑證，讓它直接和儲存服務互動。你的 application server 的角色從資料傳輸者變成存取控制者：它驗證請求、產生憑證，然後退出舞台。



不管是 AWS S3、Google Cloud Storage 還是 Azure Blob Storage，它們都支援臨時 URL，授予有時限的上傳或下載權限。CDN 也可以在提供內容之前驗證簽署過的 token。

把你的 application server 想像成演唱會場館的驗票口。當有人帶著有效票券出現，你不會親自陪他們走到座位。你驗證身份、給他們一個手環，讓他們自己找到位子。系統也是這樣，我們只驗證一次授權，然後讓客戶端直接和儲存服務互動。

簡單直傳上傳（Simple Direct Upload）

客戶端想上傳檔案時，你的伺服器不再接收整個檔案，而只接收一個「請求上傳許可」的請求。你驗證用戶、檢查配額，然後產生一個臨時的上傳 URL。這個 URL 編碼了在限定時間內（通常 15 分鐘到 1 小時）把一個特定檔案上傳到一個特定位置的權限。這就是所謂的 **Presigned URL**。

產生 Presigned URL 完全在你的 application 記憶體裡進行，不需要呼叫 Blob Storage 的網路請求。你的伺服器用它的雲端憑證建立一個簽名，儲存服務之後可以驗證這個簽名。一個 Presigned URL 長這樣：

```
<https://mybucket.s3.amazonaws.com/uploads/user123/video.mp4>
?X-Amz-Algorithm=AWS4-HMAC-SHA256
&X-Amz-Credential=AKIAIOSFODNN7EXAMPLE%2F20240115%2Fus-east-1%
&X-Amz-Date=20240115T000000Z
&X-Amz-Expires=900
&X-Amz-SignedHeaders=host
&X-Amz-Signature=b2754f5b1c9d7c4b8d4f6e9a1b2c3d4e5f6g7h8i9j0k
```

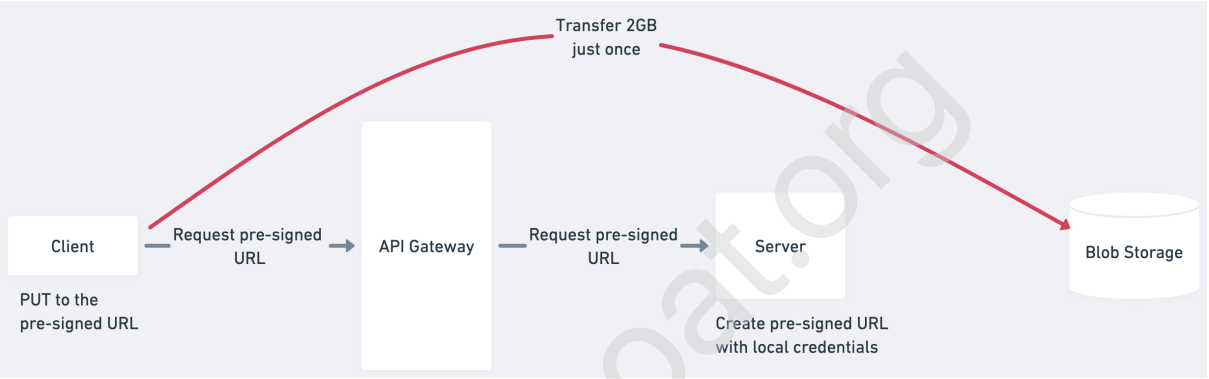
bulldmoat.org

這個簽名是請求細節（HTTP method、資源路徑、過期時間）加上你的 secret key 產生的密碼學雜湊。當客戶端上傳到這個 URL，儲存服務用它自己保存的憑證複算同一個雜湊。如果簽名吻合且 URL 尚未過期，上傳就放行。

因為任何人有了這個 URL 都能使用，你需要在產生時就把限制條件編碼進去。儲存服務讓你加上條件，在上傳時驗證：

- **content-length-range**：設定最小和最大檔案大小，防止有人在你只預期 10MB 的端點上傳 10GB
- **content-type**：確保個人頭像的端點只接受圖片，不接受影片

這些限制被烘焙進簽名裡。一個為 5MB 圖片上傳產生的 URL，如果有人試圖用它上傳 500MB 的影片，就會被拒絕。

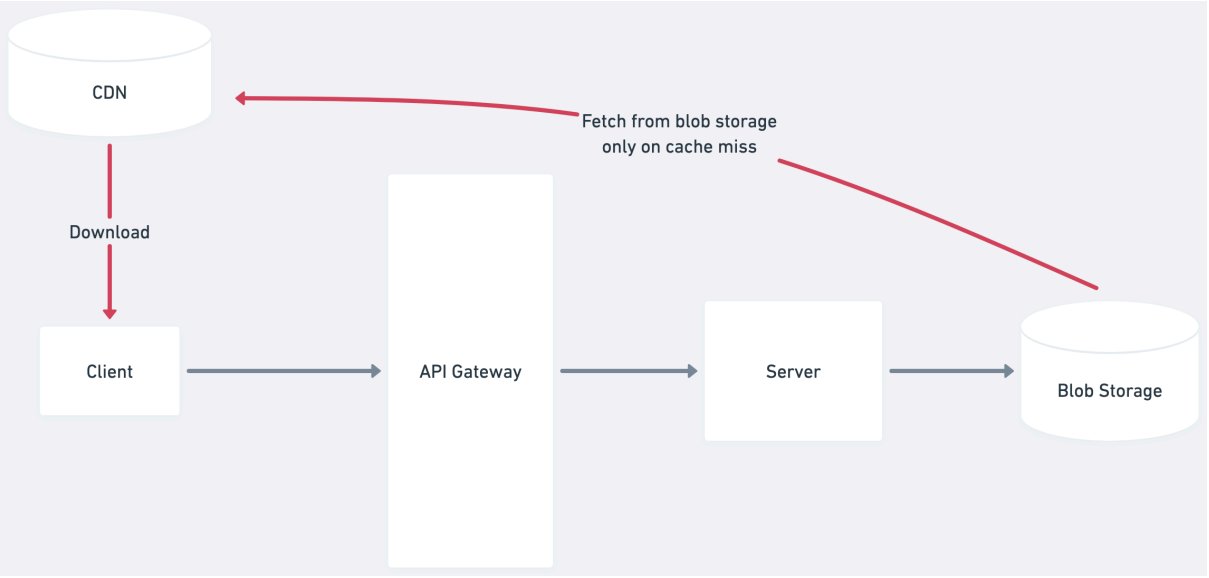


客戶端對這個 URL 發出簡單的 HTTP PUT，把檔案放在請求 body 裡。從他們的角度來看，他們直接在上傳到儲存服務。你的基礎設施從不碰那些 bytes。雪梨的用戶以全速上傳到雪梨的 Blob Storage 區域，你在維吉尼亞的伺服器可以繼續處理其他請求。

簡單直傳下載（Simple Direct Download）

下載的運作方式相同，可以直接從 Blob Storage 下載，或透過 CDN。你產生授予特定檔案臨時讀取權限的簽署 URL。直接從 Blob Storage 存取對於不頻繁的下載更簡單且便宜；CDN 分發成本較高，但透過地理分散和快取，對頻繁存取的檔案提供更好的效能。

buildnoat.org



對於 CDN 傳遞，你簽署的不是 Blob Storage 的 URL，而是 CDN 驗證的簽署 URL 或簽署 cookie。兩種方法都使用簽名，但關鍵差異在於由誰來驗證：

Blob Storage 簽名（例如 S3 Presigned URL）由儲存服務用你的雲端憑證驗證。儲存服務持有你的 secret key，可以確認你產生了這個簽名。

CDN 簽名（例如 CloudFront 簽署 URL）由 CDN 邊緣伺服器用公私鑰密碼學驗證。你持有私鑰來簽署 URL，CDN 持有對應的公鑰，在全球各地的邊緣節點驗證簽名，CDN 不需要再向你的伺服器或原始儲存服務發送請求。

大型檔案的斷點續傳（Resumable Uploads）

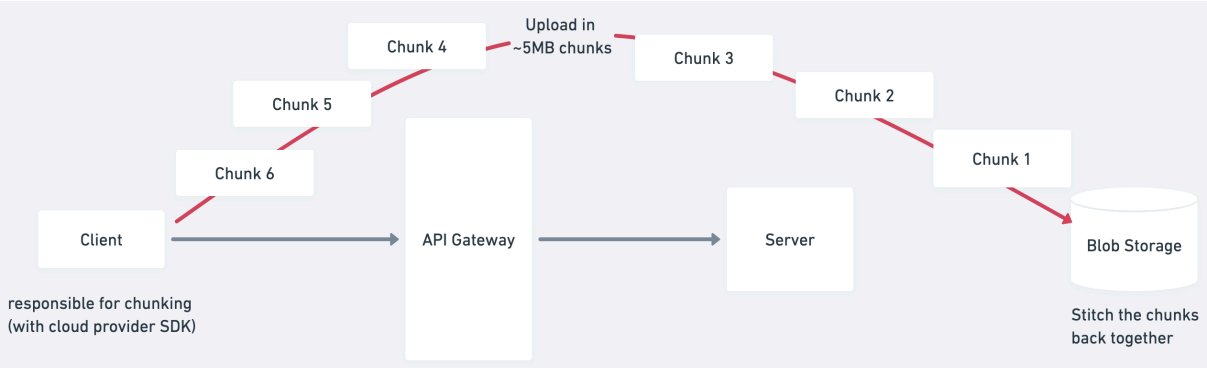
想想上傳非常大的檔案時會發生什麼。假設是一個 5GB 的影片，在一般 100 Mbps 的網路上要花 429 秒，也就是七分多鐘。如果連線在 99% 的時候斷掉，就得從頭來過。

好消息是，所有主要的雲端供應商都用分塊上傳（chunked upload）API 解決了這個問題。AWS S3 使用 Multipart Upload，每個 5MB 以上的分塊有自己的 Presigned URL；Google Cloud Storage 和 Azure 使用單一 session URL，你透過 Range header 把分塊上傳到同一個端點。

所有做法都能從失敗的分塊續傳，只是 URL 結構不同。如果出了什麼問題，你只需要重傳失敗的分塊。

客戶端上傳這些分塊時，追蹤儲存服務回傳的完成校驗和，這些只是上傳資料的雜湊，計算和驗證的成本很低。如果連線在上傳了 100 個分塊中的 60 個之後斷掉，客戶端不會從頭開始，而是查詢儲存 API 確認哪些分塊已經成功上傳，然後從第 61 塊繼續。儲存服務用 session ID（S3 的 upload ID、GCS 的 resumable upload URL 等）維護這個狀態。

buildmost.org



進度追蹤隨著分塊上傳自動產生，每個分塊完成後你就知道目前的百分比，簡單的客戶端計算就能給用戶顯示漂亮的進度條。

最後，所有分塊上傳完之後，客戶端必須呼叫一個完成端點，帶上分塊編號和校驗和的清單。儲存服務接著把這些分塊組裝成最終的物件。在這個完成步驟成功之前，儲存裡只有分塊，沒有可存取的完整檔案。未完成的 Multipart Upload 是要收費的，所以應該設定生命週期規則，在 24 到 48 小時後自動清理掉。

一旦 Multipart Upload 完成，從儲存服務的角度來看那些「分塊」就不存在了，全部合併成單一物件，就像你一次上傳了整個檔案一樣。

狀態同步的挑戰

把伺服器從傳輸路徑上移開，解決了瓶頸問題，但引入了一個全新類別的問題：分散式狀態管理。

常見的模式是把檔案的 metadata 存在資料庫，實際的檔案存在 Blob Storage。這樣既能對 metadata 做快速查詢，同時享有無限的檔案儲存空間。

以 Dropbox 這樣的檔案儲存系統為例，你的 metadata 表可能長這樣：

```
CREATE TABLE files (
    id              UUID PRIMARY KEY,
    user_id         UUID NOT NULL,
    filename        VARCHAR(255),
    size_bytes      BIGINT,
    content_type    VARCHAR(100),
    storage_key     VARCHAR(500),  -- s3://bucket/user123/fil
    status          VARCHAR(50),   -- 'pending', 'uploading',
    created_at      TIMESTAMP,
    updated_at      TIMESTAMP
);
```

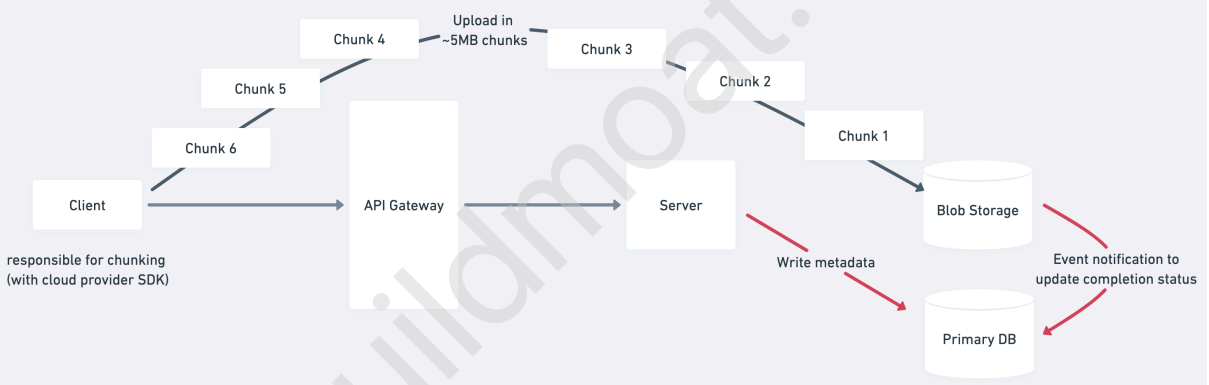
buildmoat.org

`status` 欄位告訴你一個檔案是否準備好下載，還是仍在上傳中。但在直傳的架構下，要讓這個狀態保持同步變得比較困難，因為資料庫和 Blob Storage 是在不同時間點更新的兩個獨立系統。

最簡單的做法是信任客戶端。客戶端上傳到 Blob Storage 之後，呼叫你的 API 說「上傳完成了！」，你就更新資料庫的 status。但這製造了幾個問題：

- **Race condition**：資料庫可能在檔案實際存在於 Storage 之前就顯示「completed」
- **孤兒檔案**：客戶端可能在上傳後、通知你之前崩潰，Storage 裡留下沒有資料庫記錄的檔案
- **惡意客戶端**：用戶可以在完全沒有上傳任何東西的情況下，把上傳標記為已完成
- **網路失敗**：完成通知可能根本沒有抵達你的伺服器

大多數 Blob Storage 服務用**事件通知**解決了這個問題。當 S3 接收到一個檔案，它會透過訊息服務（例如 SNS/SQS）發布事件，包含被上傳的物件的詳細資訊。事件裡帶有物件的 key，也就是你在產生 Presigned URL 時就存進資料庫的那個 `storage_key`。這讓你能找到確切的那一行來更新。



現在是儲存服務本身在確認什麼東西存在，客戶端被移出了信任鏈。但事件也可能失敗，例如網路問題、服務中斷或處理錯誤可能讓事件延遲或遺失。這就是為什麼生產系統會加上對帳（reconciliation）作為安全網：一個定期執行的任務檢查卡在 pending 狀態的檔案，並和 Storage 比對確認。

以事件作為主要的更新機制，以對帳撈起漏網之魚，你就能在不犧牲直傳效能優勢的前提下維持一致性。狀態更新的小小延遲，是不讓幾 GB 的資料流過你伺服器的合理代價。

各雲端供應商的術語對照

每家供應商對相同概念有自己的命名方式。SDK 函式和參數的確切名稱在面試中通常不需要記，但理解各家的對應關係有其價值：

buildmoat.org

功能	AWS	Google Cloud	Azure
臨時上傳 URL	Presigned URLs（PUT 或 POST）	Signed URLs（resumable 或 simple）	SAS tokens（Shared Access Signature）
分塊上傳	Multipart Upload API（5MB–5GB 每塊）	Resumable Uploads（彈性分塊大小）	Block Blobs（4MB–100MB 每塊）
事件通知	S3 Event Notifications（到 Lambda/SQS/SNS）	Cloud Storage Pub/Sub（到 Cloud Functions）	Event Grid（blob storage 事件）
附簽名 URL 的 CDN	CloudFront（signed URLs/cookies）	Cloud CDN（signed URLs）	Azure CDN（SAS tokens）
清理政策	Lifecycle Rules	Lifecycle Management	Lifecycle Management Policies

什麼時候在面試裡用這些

規則很簡單，不需要想太多：**只要你在透過 API 傳輸超過 10MB 的檔案，就應該立刻想到這個模式。** 確切的門檻取決於你的基礎設施，但 10MB 是開始感受到痛苦的地方。

常見面試情境

YouTube / 影音平台：影片上傳是最完美的使用案例。用戶上傳一個 2GB 的 4K 影片——Web App 產生 Presigned S3 URL，客戶端直接上傳到 S3（通常用 Multipart 確保可斷點續傳），S3 事件觸發轉碼工作流程。下載也是同樣的原理，——CloudFront 用簽署 URL 提供影片分段，讓自適應位元率串流成為可能，你的伺服器從不碰那些影片 bytes。

Instagram / 相片分享：相片上傳完全繞過 API server。行動 App 取得原始圖片的 Presigned URL（現代相機拍出來可能超過 50MB），直接上傳到 S3，S3 事件觸發非同步 worker 產生縮圖、套用濾鏡、提取 metadata。Feed 透過 CloudFront 搭配簽署 URL 提供圖片，防止盜連。

buildmoat.org

Dropbox / 檔案同步：檔案上傳和下載是純粹的無伺服器資料存取。你把一個 500MB 的影片拖進 Dropbox，它取得分塊上傳的 Presigned URL，直接上傳到 Blob Storage，再觸發同步工作流程。分享檔案時產生有時限的簽署 URL，讓收件人不需要有 Dropbox 帳號也能存取。

Messenger / 聊天應用：媒體分享（圖片、影片、文件）使用 Presigned URL。你在 **Messenger** 傳一段影片，它直接上傳到 Storage，聊天系統只傳遞檔案的參照。收件人取得有過期時間的簽署下載 URL，確保媒體不能被無限期存取。

什麼時候不適合用

這個模式增加了複雜度，只有在你真的面對規模或大小問題時才值得。

小檔案不需要它。 任何 10MB 以下的東西，如JSON payload、表單提交、小圖片，皆應該用一般的 API 端點。兩步驟的舞蹈在沒有實質好處的情況下增加了延遲和複雜度。你的伺服器可以處理幾千個小請求，而不需要產生 Presigned URL 的額外負擔。

需要同步驗證的場景。 如果你需要在接受上傳之前拒絕無效資料，就必須讓資料流過伺服器。例如 CSV 匯入，你必須在確認上傳之前驗證標頭和資料型別。上傳後可以非同步處理資料，但上傳過程中的即時驗證需要即時看到 bytes。

合規要求和資料檢查。 某些法規框架要求資料必須通過認證系統，或在儲存之前做掃描，例如金融服務掃描信用卡號碼、醫療系統執行 HIPAA 要求、或任何必須基於合規原因審查資料的系統。這些有限的情況下，讓資料流過你的伺服器是值得的（當然還是用分塊的方式）。

體驗要求即時回饋的場景。 如果用戶需要根據檔案內容立刻得到回饋，例如上傳大頭照後馬上看到人臉偵測結果，或文件上傳時即時產生預覽，非同步的本質會破壞使用者體驗。上傳 → 處理 → 通知的往返時間對真正互動式的功能來說太慢了。

buildmeat.org

常見的 Deep Dive 問題

「上傳到 99% 失敗了怎麼辦？」

這個問題測試你是否理解所有主要雲端供應商都提供的分塊上傳能力。當檔案超過 10MB，你應該使用分塊上傳，例如S3 Multipart Uploads（每塊 5MB 以上）、GCS Resumable Uploads（任意分塊大小）或 Azure Block Blobs（每塊 4MB 以上）。儲存服務追蹤哪些分塊已成功上傳。

連線斷掉時，客戶端不從頭開始，而是查詢哪些分塊已經上傳。像是在 S3 用 ListParts、在 GCS 查 resumable session 狀態、在 Azure 列出已提交的 block。如果分塊 1 到 19 成功但第 20 塊失敗，客戶端從第 20 塊繼續。這對連線不穩定的行動用戶或需要幾小時才能上傳的大型檔案尤其關鍵。

實作上需要客戶端追蹤上傳 session 的識別碼（S3 的 upload ID、GCS 的 resumable upload URL、Azure 的 block blob 容器 URL）。有些團隊把這個存在 localStorage，讓上傳即使在 App 重啟後也能繼續。記住，未完成的上傳是有費用的，因為雲端供應商對存放的分塊收費，所以要設定生命週期政策，在 1 到 2 天後清理掉。

「怎麼防止濫用？」

最有效的防濫用手段很簡單：不要讓用戶立刻存取他們上傳的東西。

建立一個處理流水線，讓上傳先進入隔離區（quarantine bucket）。執行病毒掃描、內容驗證和其他任何檢查，之後才把檔案移到正式的 bucket。這防止有人上傳惡意內容後立刻分享連結。

設定自動內容分析也是一個方式，當圖片識別偵測不當內容、檔案類型驗證確保「相片」不是偽裝成圖片的執行檔、大小檢查防止儲存炸彈。只有這些檢查都通過，才把檔案移到最終位置並把資料庫狀態更新為「available」。

這個做法比嘗試即時偵測濫用模式穩固得多。即使有人繞過你的 rate limiting 上傳了惡意內容，在你的系統批准之前他也用不了它。

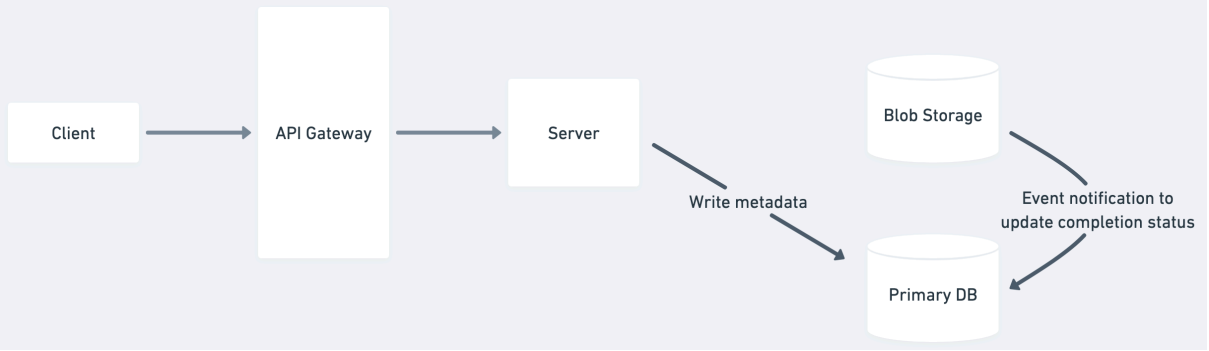
處理延遲本身也自然地抑制了濫用，像是攻擊者無法立刻知道他們的上傳是否成功，讓自動化攻擊更難執行。

永遠要在 Presigned URL 的條件裡加上檔案大小限制。沒有這個限制，有人可以用一個本來為小圖片設計的 URL 上傳幾 TB 的資料，讓你的儲存費用爆炸。

「怎麼處理 Metadata？」

檔案幾乎總是伴隨著相關的 metadata，例如上傳者是誰、用途是什麼、處理指令等等。但 Presigned URL 只是把原始 bytes 上傳到物件儲存，面試官想知道你怎麼把 metadata 和上傳的檔案關聯起來。

如前面討論過的，你的 metadata 應該存在你的主要資料庫，和檔案本身分開。挑戰在於讓兩者保持同步。當你產生 Presigned URL 時，立刻在資料庫建立狀態為 `pending` 的記錄，這個記錄包含你準備用於上傳的 storage key。這樣在檔案實際存在之前，你就有了一個參照。



buildmoat.org

有些團隊嘗試用 tag 或自訂 header 把 metadata 嵌入儲存物件本身。S3 讓你附上最多 10 個 tag，每個最多 256 個字元。但這很受限，而且讓查詢變得很痛苦，因為你無法靠掃描物件 tag 有效率地找出「用戶 X 在過去一週上傳的所有 PDF」。把豐富的 metadata 放在它該在的地方：你的資料庫。

Storage key 是你的連接點。用一個一致的模式，例如 `uploads/{user_id}/{timestamp}/{uuid}`，包含有用的資訊但防止衝突。儲存事件觸發時會帶上這個 key，讓你找到確切的資料庫行來更新。永遠不要讓客戶端自己指定 key，這會帶來覆蓋風險和安全問題。

對於必須和上傳操作原子性同步的關鍵 metadata，你可以用條件上傳，比如產生要求特定 metadata header 的 Presigned URL，如果客戶端沒有帶上這些 header，上傳就失敗。但這增加了複雜度，而大多數使用案例不需要這種程度的嚴格性。

「怎麼確保下載夠快？」

直接從 Blob Storage 下載可行，但錯過了巨大的優化機會。雪梨的用戶從維吉尼亞的 Blob Storage 下載，每個請求還是有 200ms 的延遲。對一個 100MB 的檔案可能還好；對幾千個小檔案或互動式應用，就不行了。

CDN 透過在全球各地快取內容來解決地理問題。當你產生簽署的 CloudFront URL，第一個用戶從源站拉取，但那個地區之後的用戶從快取的副本取得，延遲只有個位數毫秒。200ms 和 5ms 的差異看起來不大，但乘以幾百個請求就很可觀了。

但 CDN 本身對大型檔案下載沒有幫助，一個 5GB 的檔案還是要花幾分鐘，連線斷了還得重新開始。解法是 **Range Request**——HTTP 讓你只下載檔案特定 byte 範圍的能力：

```
GET /large-file.zip
Range: bytes=0-10485759   (前 10MB)
```

這讓下載可以斷點續傳。追蹤哪些範圍已完成，重新連線後只請求遺失的部分。現代瀏覽器和下載管理員，只要你的 Storage 和 CDN 支援 Range Request（它們都支援），會自動處理這件事。你只需要確保你的簽署 URL 沒有限制必要的 HTTP 動詞或 header。

對於極端情況，例如分發大型資料集或遊戲資源，有些團隊實作平行分塊下載，把檔案拆成幾塊，同時下載四到六塊，再在客戶端重組。這可以讓下載速度提升三到四倍，繞過每個連線的限速。但這種複雜度通常不值得，大多數用戶受限於總頻寬，而不是每個連線的限制。

buildmeat.org

務實的做法是：透過 CDN 提供所有內容並設定合適的快取 header；確保大型檔案可以使用 Range Request；讓 CDN 和瀏覽器做優化。只有在你在分發幾 GB 的大型檔案且用戶在良好網路下仍抱怨速度時，才考慮平行下載這類進階手段。

總結

大型檔案處理是一個在幾乎所有涉及用戶生成內容的系統設計面試中都會出現的模式。當你聽到「影片上傳」、「檔案分享」或「相片儲存」，立刻想到繞過你的伺服器來做實際的資料傳輸。

核心洞察是從「讓 bytes 流過你的基礎設施」轉移到「編排存取權限並管理分散式狀態」。

在面試中，展示你理解效能優勢和複雜度代價兩面。說清楚什麼時候用直傳（超過 10MB），什麼時候不用（小檔案、合規要求）。最後，別忘了狀態同步的挑戰，這是許多面試者會卡關的地方。

buildmoat.org